

Intro to Linux:

Navigation:

ls -l # long format

ls -a # include hidden files

ls -lh # human readable sizes

cd ~ # home directory

File Operations:

cp src dest # copy file

mv src dest # move/rename file

rm -r dir # delete directory

touch file # create empty file

Viewing Files:

less file # scrollable view

head file # first 10 lines

tail file # last 10 lines

tail -f file # live updates

Searching:

grep "text" file # search text in file

grep -r "text" dir # recursive search dir

find dir -name file # find by name

Find / -name file 2>/dev/null # search entire system

which command # path of command

Pipes & Redirection:

command1 | command2 # pipe output

> file # overwrite file

>> file # append output to file

< file # file as input to command

2> file # redirect errors to file

EX: cat file.txt | grep flag # print only lines containing "flag"

Permissions:

ls -l #shows permissions

-rwxr-xr-- # EX permission

- r(read), w(write), x(execute)
- Command Format = type(-)
owner(rwx) group(r-x) others (r--)

Change Permissions:

chmod 755 file # Owner=rwx Group=rx Others=rx

chmod +x file # Add execute permission

chmod u+r file # Add read permission to owner

chmod g-w file # Remove write permission from group

Change Ownership:

chown user file # Change file owner

chown user:group file # Change owner and group

Processes:

ps # show running processes

ps aux # detailed process list (a=all users, u = user format, x = include background process)

kill PID # terminate process (by id)

kill -9 PID # force kill process

Environment Variables:

env # show environment variables

echo \$PATH #print path variable(where system looks for commands)

Computing 101

Number Systems:

Binary # Base 2 (0,1)

Decimal # Base 10 (normal numbers)

Hexadecimal # Base 16 (0–9 A–F)

ASCII # maps numbers to characters

Memory Basics:

Stack # stores function calls and local variables(pop, push)

Heap #dynamically allocated memor (malloc)

Programs & Arguments:

EX: ./program arg1 arg2

Memory Addressing Syntax:

Indirect Addressing #Mov rax, [rbx] # move value stored at mem address in rbx to rax

Direct addressing #Mov rax, [0x400] #load value from memory address 0x400

Offset/Base addressing #mov rax,
[rbx+8] #rax = value at memory address =
rbx+8

_Workings of the Web

URL Structure:

protocol://host/path?query=value

EX: <https://example.com/page?id=5>

Parts:

- Protocol #HTTP or HTTPS
- Host #domain name
- Path # location of source
- Query # parameters sent to server

HTTP Request Format:

GET /page HTTP/1.1 # request line

Host: example.com # required header

Header: value # additional headers

Common HTTP Methods:

GET # retrieve resource

POST # send data to server\

PUT # update resource

DELETE # remove resource

Using curl:

curl <https://example.com> # send GET
request

curl -X POST <https://site.com> # send
POST request

curl -d "user=admin" <https://site.com> #
send POST data

HTTP Headers:

User-Agent # Browser identification

Cookie # Session information

Content-Type # Format of data sent

Authorization # Authentication info

Headers control how servers respond

Web Security:

Parameter Tampering:

Changing values in URL's to manipulate
server behavior

EX site.com/admin?id=5 change id=1 to
access different data

Cookie Manipulation:

Cookies store session or authentication
data

EX: Cookie: session=abc123

Editing cookies may allow for privilege
escalation and session hijacking

Cross-site Request Forgery (CSRF):

Occurs when a victims browser sends a
request to a website while already
authenticated, causing an unintended
action.

Browser trusts site with cookies, server
cannot distinguish request origin.

CSRF Tokens # server requires token in
requests i.e <input type="hidden"
name='csrf_token' value="A82KF39">

SQL Injection:

When user input is inserted into SQL query

**EX: SELECT * FROM users WHERE
name='INPUT'** # effect: returns all
users(bypass login)

Cross-Site Scripting (XSS):

Inject javascript into webpage input

EX: <script>alert(1)</script>

Types:

- Stored # saved on server
- Reflected # returned in response
- DOM-based #executed in browser
JS

Useful Tools for Web Challenges:

curl #send manual HTTP requests

Browser devtools # inspect network traffic

View-source # view webpage HTML

Check For:

- Headers
- Cookies
- URL parameters
- responses

